1. **(Query formation)** From the current work token, a fixed affine (or simple FFN) subroutine writes a query vector $q_{t,r}$ into the query register for read $r \in \{1, \ldots, R_t\}$.

2. **(Routing read)** A single attention call reads from the prompt slots into the work token via a residual add:

$$z \leftarrow z + \mathrm{Inject}\big(\mathcal{R}_\tau(q_{t,r}; P)\big),$$

   where $\mathrm{Inject} : \mathbb{R}^D \to \mathbb{R}^D$ is a fixed linear map selecting the coordinates/registers that receive the read vector (implemented by W_V and subsequent fixed linear maps).

3. **(Assembly)** After R_t reads, a fixed linear unpacking produces the assembled fragment $\widehat{\lambda}_t \in \mathbb{R}^{r_t}$ inside registers of the work token.

4. **(Arithmetic update)** A token-wise FFN applies an update to the work token so that the macro-state register s_t is updated to s_{t+1}.

5. **(Optional write-back)** In variants with $\mathcal{I}_{\mathrm{wrk}} \neq \emptyset$, one additional attention call writes a bounded summary into a selected work-memory slot. (This step can be omitted in the simplest construction.)

We emphasize that the *only task-dependent object* is the prompt P (hence K_P, V_P). All subroutines above are realized by fixed weights in θ^*.

Abstract update form. Let $\lambda_t(p)$ denote the *ideal* parameter fragment for macro-step t (as stored in $\mathcal{I}_{\mathrm{par}}$), and let Ψ_t denote the *ideal* macro-update. The intended macro-dynamics is

$$s_{t+1} \;=\; \Psi_t\big(s_t, \lambda_t(p)\big), \qquad t = 0, \ldots, T_{\mathrm{macro}} - 1,$$

as in Appendix E. The realized executor produces $\widehat{s}_t$ and $\widehat{\lambda}_t$ with bounded errors controlled by Appendices C and D.

G.3. Execution invariants

We now state the invariants maintained for all macro-steps $t \in \{0, \ldots, T_{\mathrm{macro}}\}$. All statements below are uniform over $P \in \mathcal{P}_{\mathrm{adm}}$ and $x \in \mathcal{X}$.

Inv-1 (Uniform boundedness). There exists a bound $B > 0$ depending only on the fixed executor architecture and construction constants (in particular on θ^*, L, D, T_{macro}, and the encoding ranges defining $\mathcal{P}_{\mathrm{adm}}$), such that all designated token vectors remain in the box $\mathcal{B}_B = [-B, B]^D$:

$$\|z_i^{(t)}\|_\infty \leq B \quad \text{for all } i \in \mathsf{PROMPT} \cup \{i_x, i_{\mathrm{wrk}}, i_{\mathrm{out}}\} \text{ and all macro-times } t.$$

In particular, prompt tokens satisfy $\|p_j\|_\infty \leq B$ for all $j \in [L]$, and since the value map W_V is fixed in θ^*, the prompt values $(V_P)_{j,:}$ are also uniformly bounded (up to a constant factor absorbed into B), as required by the routing bound.

Inv-2 (Prompt slot partition). The prompt slots satisfy the partition

$$[L] = \mathcal{I}_{\mathrm{par}} \,\dot\cup\, \mathcal{I}_{\mathrm{wrk}}$$

as in Section 4.2, and all records needed by the program are stored in $\mathcal{I}_{\mathrm{par}}$ using the key–value encoding of Appendix F.

Inv-3 (Addressability / margin). For each routing read (t, r) there exists a target label $\ell_{t,r}$ (hence a target slot $j(\ell_{t,r}) \in \mathcal{I}_{\mathrm{par}}$) such that the query $q_{t,r}$ satisfies a score margin $\Delta_{t,r} > 0$:

$$\langle q_{t,r}, k_{j(\ell_{t,r})} \rangle \;\geq\; \langle q_{t,r}, k_j \rangle + \Delta_{t,r} \quad \text{for all } j \in [L] \setminus \{j(\ell_{t,r})\}.$$

Inv-4 (Register integrity). Each subroutine only writes to intended registers: query formation only changes Π_{qry}, routing injection only affects designated payload registers, and the arithmetic FFN updates Π_{st} according to the intended Ψ_t while preserving boundedness (consistent with Inv-1).

G.4. Maintaining Inv-1: a generic sufficient condition

The next lemma records a clean sufficient condition under which the uniform boundedness invariant (Inv-1) holds. Its role is to align the boundedness bookkeeping with the actual executor dynamics (which includes residual additions).

Lemma G.1 (Sufficient condition for Inv-1)**.** *Assume that the construction fixes $B_0 > 0$ such that:*

(i) The encoding maps used to form the initial tokens $(P, u(x), w, o)$ produce vectors in $\mathcal{B}_{B_0}$ uniformly over $x \in \mathcal{X}$ and all $P \in \mathcal{P}_{\mathrm{adm}}$.

(ii) Each macro-step subroutine (query formation, routing injection, linear assembly, and arithmetic FFN) is implemented by fixed weights chosen so that $\mathcal{B}_{B_0}$ is an invariant set for the executor dynamics, i.e., each subroutine maps inputs in $\mathcal{B}_{B_0}$ to outputs in $\mathcal{B}_{B_0}$. (For instance, this can be ensured by range-restricted encodings together with appropriate scaling of linear maps so that residual additions remain within $\mathcal{B}_{B_0}$ on that set; explicit clipping/saturation is one sufficient but not necessary mechanism.)

Then Inv-1 holds with $B = B_0$.

Proof. By induction over macro-steps. The base case follows from (i). For the inductive step, assume all designated tokens lie in $\mathcal{B}_{B_0}$ at macro-time t. Applying the subroutines of macro-step t preserves membership in $\mathcal{B}_{B_0}$ by (ii), hence the designated tokens remain bounded at macro-time $t + 1$. □

G.5. Maintaining invariants and deriving per-step bounds

The next lemma shows that Inv-3 implies an exponentially small routing error for each read, and that Inv-1 guarantees that the arithmetic approximation can be taken uniformly on a fixed compact domain.

Lemma G.2 (Per-read routing and per-step arithmetic prerequisites)**.** *Assume Inv-1–Inv-3 hold at macro-step t. Let $q_{t,r}$ be a query with margin $\Delta_{t,r}$ for its target slot. Then the routed read obeys*

$$\left\|\mathcal{R}_\tau(q_{t,r}; P) - (V_P)_{j(\ell_{t,r}),:}\right\|_\infty \;\le\; 2B\,(L-1)e^{-\Delta_{t,r}/\tau}.$$

Moreover, by Inv-1, all token-wise FFN approximations invoked in the arithmetic step can be taken uniformly over $\mathcal{B}_B = [-B, B]^D$.

Proof. The routing bound is exactly Lemma C.2 (Appendix C). The arithmetic prerequisite follows since Inv-1 ensures that all FFN inputs remain in $\mathcal{B}_B$, the compact domain on which Appendix D provides uniform approximation. □

G.6. A generic macro-step error statement

We now state the executor-level conclusion in a form that plugs into the composition template.

Proposition G.3 (Macro-step realization with budgeted errors)**.** *Fix macro-step t and suppose Inv-1–Inv-4 hold. Assume that:*

- *Each routing read (t, r) satisfies a margin $\Delta_{t,r}$ and uses temperature τ, so that the total assembled parameter fragment $\widehat{\lambda}_t$ obeys*
$$\|\widehat{\lambda}_t - \lambda_t(p)\|_\infty \le \delta_t^{\mathrm{route}}$$
for some prescribed $\delta_t^{\mathrm{route}}$ (obtained by summing the per-read bounds and the linear assembly operator norm).
- *The token-wise FFN used in the arithmetic step approximates the ideal macro-update $(s, \lambda) \mapsto \Psi_t(s, \lambda)$ to accuracy $\delta_t^{\mathrm{arith}}$ uniformly over the bounded state set $\mathcal{S} \subseteq \mathbb{R}^m$ and admissible λ.*

Then the work-state register satisfies the one-step bound

$$\|\widehat{s}_{t+1} - s_{t+1}\|_\infty \;\le\; \|\widehat{s}_t - s_t\|_\infty \;+\; \delta_t^{\mathrm{arith}} \;+\; L_t\,\delta_t^{\mathrm{route}},$$

where L_t is a Lipschitz constant of Ψ_t in its parameter argument (as in Appendix E).

Proof. This is a direct instantiation of Lemma E.1 in Appendix E. The only additional ingredient is that the realized fragment $\widehat{\lambda}_t$ is produced by finitely many reads followed by fixed linear assembly; hence its deviation from $\lambda_t(p)$ is bounded by the sum of the individual routing errors multiplied by the assembly operator norm. □

G.7. End-to-end view

Combining Proposition G.3 over $t = 0, \ldots, T_{\text{macro}} - 1$ and using the budget allocation in Appendix E yields a uniform executor emulation bound

$$\sup_{x \in \mathcal{X}} \left| F_{\theta^*}(p, x) - \widetilde{F}_{\theta^*}(p, x) \right| \;\le\; \varepsilon_{\text{exec}},$$

where $\widetilde{F}_{\theta^*}$ denotes the idealized program output under perfect routing and perfect arithmetic. This is the executor-side half of the triangle-inequality split used in the main proof.

Remark (what must be specified in the main construction). To fully instantiate this appendix for the final proof, the main construction must specify: (i) the label set $\{\ell_{t,r}\}$ and memory layout in $\mathcal{I}_{\text{par}}$, (ii) how queries $q_{t,r}$ are generated and why they satisfy the margin condition (Inv-3), and (iii) why the uniform boundedness (Inv-1) holds with a constant B uniform over all admissible prompts in $\mathcal{P}_{\text{adm}}$, under the chosen encodings, injections, and arithmetic updates. These are precisely the points addressed by the routing, encoding, arithmetic, and composition appendices.

H. Full Proof of the Main Theorem

In this appendix we give a complete proof of the main theorem stated in the main text. This section turns the proof sketch into a fully detailed argument by explicitly carrying out the approximation split, the prompt-encoding step, and the executor emulation bound, and then combining them into the final uniform error guarantee.

Theorem H.1 (Main theorem (prompt switching with a fixed executor))**.** *Let $\mathcal{X} \subset \mathbb{R}^d$ be compact as in Assumption 3.1. There exists a fixed set of executor parameters θ^* (a simplified Transformer as in Appendix B) such that for every $g \in C(\mathcal{X})$ and every $\varepsilon > 0$ there exists a soft prompt $p_g \in \mathcal{P}$ with*

$$\sup_{x \in \mathcal{X}} \left| F_{\theta^*}(p_g, x) - g(x) \right| \;<\; \varepsilon.$$

Equivalently, for the prompt-induced family $H_{\theta^*} = \{F_{\theta^*}(p, \cdot) : p \in \mathcal{P}\}$ (Definition 3.4), we have $\overline{H_{\theta^*}}^{\|\cdot\|_\infty} = C(\mathcal{X})$.

Proof. Fix an arbitrary target $g \in C(\mathcal{X})$ and $\varepsilon > 0$. Set

$$\varepsilon_{\text{approx}} := \varepsilon/2, \qquad \varepsilon_{\text{exec}} := \varepsilon/2, \qquad \text{so that } \varepsilon_{\text{approx}} + \varepsilon_{\text{exec}} = \varepsilon.$$

Step 1: Classical approximation by a small ReLU MLP. By the classical universal approximation theorem for ReLU networks on compact domains, there exists a (finite) ReLU MLP $N_g : \mathcal{X} \to \mathbb{R}$ such that

$$\sup_{x \in \mathcal{X}} |N_g(x) - g(x)| \;\le\; \varepsilon_{\text{approx}}. \tag{14}$$

Without loss of generality, choose N_g to be a depth-$S = 3$ (one-hidden-layer) ReLU MLP in the sense of Definition 7.1. In particular, write

$$N_g(x) = a^\top \sigma(Wx + b) + c \quad \text{and identify} \quad W^{(1)} = W,\ b^{(1)} = b,\ W^{(2)} = a^\top,\ b^{(2)} = c.$$

Collect the parameters in the same format as Definition 7.1:

$$\pi_g := \big(W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}\big),$$

and define the (finite) bound

$$\Lambda := \max\Big\{\|W^{(1)}\|_{\max}, \|b^{(1)}\|_{\max}, \|W^{(2)}\|_{\max}, |b^{(2)}|\Big\} < \infty.$$

Thus N_g belongs to the bounded class in Definition 7.1 for this (g, ε)-dependent Λ, and (14) holds.